Date.prototype.toLocaleTimeString()

📄 Título

Date.prototype.toLocaleTimeString()

📝 Definición

`Date.prototype.toLocaleTimeString()` es un método de JavaScript que devuelve una **representación en string** de la parte de tiempo (hora) de un objeto `Date`, formateada según la **configuración regional** (idioma y país) especificada.

🧠 ¿Qué es?

Es el método estándar para mostrar horas en un formato **comprensible para el usuario**, teniendo en cuenta su **cultura y región**.

A diferencia de `toTimeString()` (que usa el formato fijo del navegador), `toLocaleTimeString()` permite personalizar el formato y adaptarlo al idioma y costumbres locales del usuario.

El método utiliza el sistema de **internacionalización** (i18n) de JavaScript para formatear horas correctamente, mostrando:

- **Formato de 12 o 24 horas** según la región (ej: 3:30 PM en EE.UU., 15:30 en España).
- **Nombres de zonas horarias** localizados.
- **Separadores** apropiados (:, ., etc.).
- **Numerales** en el sistema de escritura local.

🎯 ¿Para qué sirve?

- Mostrar horas en **interfaces de usuario** con formato local.
- Crear aplicaciones **internacionalizadas** (i18n) con soporte multi-idioma.
- Generar horas legibles para **informes, logs y registros**.
- Formatear horas según las **preferencias del usuario**.
- Mostrar horas en **calendarios, agendas y sistemas de citas**.
- Presentar **marcas de tiempo** en aplicaciones de mensajería.
- Mostrar **horarios de eventos y citas**.
- Crear **dashboards con métricas temporales**.

- Implementar **relojes y temporizadores** en diferentes idiomas.

⚙️ ¿Cómo funciona?

El método utiliza el **locale** (configuración regional) y las **opciones** proporcionadas para formatear la hora.

Proceso:

1. Determina el locale a usar (por parámetro o del sistema).
2. Aplica las opciones de formato (si se proporcionan).
3. Genera la representación en string según las reglas culturales.
4. Devuelve el string formateado.

Importante: `toLocaleTimeString()` devuelve **solo la hora**, sin la fecha. Para fecha y hora usar `toLocaleString()`.

📝 Sintaxis / Estructura (JavaScript)

```
date.toLocaleTimeString([locales[, options]])
```

Parámetros:

- `locales` (opcional): String o array de strings con códigos de idioma/región.
- `options` (opcional): Objeto con opciones de formato.

Opciones principales:

- `timeStyle`: `'full' | 'long' | 'medium' | 'short'`
- `timeZone`: Zona horaria (ej: `'UTC'`, `'America/New_York'`)
- `hour`: `'numeric' | '2-digit'`
- `minute`: `'numeric' | '2-digit'`
- `second`: `'numeric' | '2-digit'`
- `hour12`: `true | false` (formato 12h o 24h)
- `timeZoneName`: `'long' | 'short'` (nombre de la zona horaria)

💡 Ejemplos

```
// 1. Uso básico - sin opciones
const date = new Date(2024, 11, 25, 15, 30, 45);
```

```
console.log(date.toLocaleTimeString()); // Según la configuración de
l sistema
// En España: "15:30:45"
// En EE.UU.: "3:30:45 PM"

// 2. Especificar locale
console.log(date.toLocaleTimeString('es-ES')); // "15:30:45"
console.log(date.toLocaleTimeString('en-US')); // "3:30:45 PM"
console.log(date.toLocaleTimeString('de-DE')); // "15:30:45"
console.log(date.toLocaleTimeString('ja-JP')); // "15:30:45"
console.log(date.toLocaleTimeString('fr-FR')); // "15:30:45"

// 3. Estilos predefinidos (timeStyle)
console.log(date.toLocaleTimeString('es-ES', { timeStyle: 'short'
}));
// "15:30"

console.log(date.toLocaleTimeString('en-US', { timeStyle: 'medium'
}));
// "3:30:45 PM"

console.log(date.toLocaleTimeString('es-ES', { timeStyle: 'long'
}));
// "15:30:45 GMT+1"

console.log(date.toLocaleTimeString('en-US', { timeStyle: 'full'
}));
// "3:30:45 PM GMT+01:00"

// 4. Formato personalizado con opciones
const opcionesPersonalizadas = {
  hour: '2-digit',
  minute: '2-digit',
  second: '2-digit'
};

console.log(date.toLocaleTimeString('es-ES', opcionesPersonalizada
s));
// "15:30:45"

console.log(date.toLocaleTimeString('en-US', opcionesPersonalizada
s));
// "03:30:45 PM"
```

```
// 5. Formato 12 vs 24 horas
console.log(date.toLocaleTimeString('en-US', {
  hour: 'numeric',
  minute: 'numeric',
  hour12: true
}));
// "3:30 PM"

console.log(date.toLocaleTimeString('es-ES', {
  hour: 'numeric',
  minute: 'numeric',
  hour12: false
}));
// "15:30"

// 6. Incluir zona horaria
console.log(date.toLocaleTimeString('es-ES', {
  hour: '2-digit',
  minute: '2-digit',
  timeZoneName: 'short'
}));
// "15:30 GMT+1"

console.log(date.toLocaleTimeString('en-US', {
  hour: '2-digit',
  minute: '2-digit',
  timeZoneName: 'long',
  hour12: true
}));
// "03:30 PM Central European Standard Time"

// 7. Diferentes zonas horarias
const fechaUTC = new Date('2024-12-25T15:30:45Z');

console.log(fechaUTC.toLocaleTimeString('es-ES', {
  timeZone: 'UTC',
  hour: '2-digit',
  minute: '2-digit'
}));
// "15:30"

console.log(fechaUTC.toLocaleTimeString('es-ES', {
  timeZone: 'America/New_York',
  hour: '2-digit',
```

```
  minute: '2-digit'
}));
// "10:30" (UTC-5)

console.log(fechaUTC.toLocaleTimeString('es-ES', {
  timeZone: 'Asia/Tokyo',
  hour: '2-digit',
  minute: '2-digit'
}));
// "00:30" (UTC+9, día siguiente)

// 8. Sin segundos ni minutos
console.log(date.toLocaleTimeString('es-ES', {
  hour: '2-digit'
}));
// "15"

// 9. Múltiples locales (fallback)
console.log(date.toLocaleTimeString(['es-ES', 'en-US']));
// Usa es-ES si está disponible, sino en-US

// 10. Formato en diferentes regiones
const times = [
  date.toLocaleTimeString('es-ES'),
  date.toLocaleTimeString('en-US'),
  date.toLocaleTimeString('de-DE'),
  date.toLocaleTimeString('fr-FR'),
  date.toLocaleTimeString('it-IT'),
  date.toLocaleTimeString('ja-JP'),
  date.toLocaleTimeString('pt-BR'),
  date.toLocaleTimeString('ru-RU'),
  date.toLocaleTimeString('ar-SA')
];

console.log(times);
// ["15:30:45", "3:30:45 PM", "15:30:45", ...]

// 11. Reloj en tiempo real
function startClock(locale = 'es-ES') {
  function updateClock() {
    const now = new Date();
    const timeString = now.toLocaleTimeString(locale, {
      hour: '2-digit',
      minute: '2-digit',
```

```
      second: '2-digit',
      hour12: false
    });
    console.log(timeString);
  }

  updateClock();
  setInterval(updateClock, 1000);
}

// startClock('es-ES');

// 12. Formateo de duración (solo horas y minutos)
function formatDuration(minutes) {
  const hours = Math.floor(minutes / 60);
  const mins = minutes % 60;
  const date = new Date(0, 0, 0, hours, mins);

  return date.toLocaleTimeString('es-ES', {
    hour: '2-digit',
    minute: '2-digit',
    hour12: false
  });
}

console.log(formatDuration(90)); // "01:30"
console.log(formatDuration(135)); // "02:15"

// 13. Comparación de formatos
function compareTimeFormats(date, locales) {
  const results = {};
  locales.forEach(locale => {
    results[locale] = date.toLocaleTimeString(locale, {
      timeStyle: 'full'
    });
  });
  return results;
}

const locales = ['es-ES', 'en-US', 'de-DE', 'fr-FR', 'ja-JP'];
console.log(compareTimeFormats(new Date(), locales));

// 14. Para interfaces de usuario
function formatTimeRange(startTime, endTime, locale = 'es-ES') {
```

```
  const start = new Date(startTime);
  const end = new Date(endTime);

  const options = {
    hour: '2-digit',
    minute: '2-digit'
  };

  return {
    start: start.toLocaleTimeString(locale, options),
    end: end.toLocaleTimeString(locale, options),
    range: `${start.toLocaleTimeString(locale, options)} -${end.toLo
caleTimeString(locale, options)}`
  };
}

const range = formatTimeRange('2024-12-25T10:00:00', '2024-12-25T12:
30:00', 'es-ES');
console.log(range);
// { start: "10:00", end: "12:30", range: "10:00 - 12:30" }

// 15. Conversión a formato ISO local
function toLocalISOString(date) {
  const options = {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit'
  };
  return date.toLocaleString('es-ES', options).replace(/\//g, '-');
}

console.log(toLocalISOString(new Date()));
// "25-12-2024 15:30:45"
```

⚠️ Errores comunes

- **No especificar locale**: El formato puede variar según la configuración del sistema.
- **Confundir** `toLocaleTimeString()` **con** `toLocaleString()`: El primero solo muestra hora, el segundo fecha y hora.
- **Asumir formato de 12h siempre**: Depende del locale y las opciones.

- **No manejar zonas horarias**: Las horas se convierten a la zona horaria local por defecto.
- **Pasar strings como fecha sin convertir**: Usar `new Date()` primero.
- **Olvidar que los meses empiezan en 0**: `new Date(2024, 11, 25, 15, 30)` es diciembre, no noviembre.

💼 Casos de uso

1. **Sistema de mensajería con marcas de tiempo**:

```
class MessageSystem {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.messages = [];
  }

  sendMessage(user, text) {
    const date = new Date();
    this.messages.push({
      user,
      text,
      timestamp: date,
      formatted: date.toLocaleTimeString(this.locale, {
        hour: '2-digit',
        minute: '2-digit'
      })
    });
  }

  getMessages() {
    return this.messages.map(msg => ({
      ...msg,
      timeAgo: this.getTimeAgo(msg.timestamp)
    }));
  }

  getTimeAgo(date) {
    const diff = Math.floor((new Date() - date) / 1000);
    if (diff < 60) return 'hace unos segundos';
    if (diff < 3600) return `hace${Math.floor(diff / 60)} minutos
`;
    if (diff < 86400) return `hace${Math.floor(diff / 3600)} hora
s`;
    return date.toLocaleDateString(this.locale);
```

```
  }
}

const chat = new MessageSystem('es-ES');
chat.sendMessage('Ana', '¡Hola!');
chat.sendMessage('Carlos', '¿Cómo estás?');
console.log(chat.getMessages());
```

2. **Dashboard de actividad en tiempo real**:

```
class ActivityDashboard {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.activities = [];
    this.updateInterval = null;
  }

  logActivity(type, details) {
    const now = new Date();
    this.activities.unshift({
      type,
      details,
      timestamp: now,
      time: now.toLocaleTimeString(this.locale, {
        hour: '2-digit',
        minute: '2-digit',
        second: '2-digit'
      })
    });

    // Mantener solo últimas 100 actividades
    if (this.activities.length > 100) {
      this.activities.pop();
    }
  }

  getActivities() {
    return this.activities;
  }

  startLiveUpdate() {
    this.updateInterval = setInterval(() => {
      this.logActivity('heartbeat', { status: 'active' });
```

```
    }, 60000);
  }

  stopLiveUpdate() {
    if (this.updateInterval) {
      clearInterval(this.updateInterval);
    }
  }
}

const dashboard = new ActivityDashboard('es-ES');
dashboard.logActivity('login', { user: 'admin' });
dashboard.logActivity('page_view', { page: 'dashboard' });
console.log(dashboard.getActivities());
```

3. **Sistema de reservas con horarios**:

```
class BookingSystem {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.bookings = [];
  }

  createBooking(user, date, duration = 60) {
    const start = new Date(date);
    const end = new Date(start.getTime() + duration * 60000);

    const timeOptions = {
      hour: '2-digit',
      minute: '2-digit'
    };

    const booking = {
      user,
      start: start,
      end: end,
      startFormatted: start.toLocaleTimeString(this.locale, timeO
ptions),
      endFormatted: end.toLocaleTimeString(this.locale, timeOptio
ns),
      duration: duration,
      timeRange: `${start.toLocaleTimeString(this.locale, timeOpt
ions)} -${end.toLocaleTimeString(this.locale, timeOptions)}`,
```

```
      dateFormatted: start.toLocaleDateString(this.locale, {
        weekday: 'long',
        day: 'numeric',
        month: 'long',
        year: 'numeric'
      })
    };

    this.bookings.push(booking);
    return booking;
  }

  getTodayBookings() {
    const today = new Date().toDateString();
    return this.bookings
      .filter(b => b.start.toDateString() === today)
      .sort((a, b) => a.start - b.start)
      .map(b => ({
        ...b,
        timeDisplay: b.timeRange,
        dateDisplay: b.dateFormatted
      }));
  }
}

const calendar = new BookingSystem('es-ES');
calendar.createBooking('Juan', '2024-12-25T10:00:00', 90);
calendar.createBooking('María', '2024-12-25T14:30:00', 60);
console.log(calendar.getTodayBookings());
```

4. **Conversor de zonas horarias**:

```
class TimeZoneConverter {
  constructor(locale = 'es-ES') {
    this.locale = locale;
  }

  convertTime(date, fromTimeZone, toTimeZone) {
    const options = {
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
      timeZone: toTimeZone,
```

```
      hour12: false
    };

    const dateObj = new Date(date);
    const result = dateObj.toLocaleTimeString(this.locale, option
s);

    return {
      original: dateObj.toLocaleTimeString(this.locale, {
        ...options,
        timeZone: fromTimeZone
      }),
      converted: result,
      fromZone: fromTimeZone,
      toZone: toTimeZone,
      date: dateObj.toLocaleDateString(this.locale, {
        timeZone: toTimeZone
      })
    };
  }

  getTimeZoneList() {
    return Intl.supportedValuesOf('timeZone');
  }
}

const converter = new TimeZoneConverter('es-ES');
const result = converter.convertTime(
  '2024-12-25T15:30:00Z',
  'UTC',
  'America/New_York'
);
console.log(result);
```

5. **Reloj con diferentes formatos**:

```
function createClock(element, locale = 'es-ES', format = 'full')
{
  let interval = null;

  function update() {
    const now = new Date();
    let options;
```

```
    switch(format) {
      case 'full':
        options = {
          hour: '2-digit',
          minute: '2-digit',
          second: '2-digit'
        };
        break;
      case 'compact':
        options = {
          hour: '2-digit',
          minute: '2-digit'
        };
        break;
      case '12h':
        options = {
          hour: 'numeric',
          minute: '2-digit',
          hour12: true
        };
        break;
      case '24h':
        options = {
          hour: '2-digit',
          minute: '2-digit',
          hour12: false
        };
        break;
      case 'withSeconds':
        options = {
          hour: '2-digit',
          minute: '2-digit',
          second: '2-digit',
          hour12: false
        };
        break;
    }

    element.textContent = now.toLocaleTimeString(locale, option
s);
  }

  function start() {
```

```
    if (interval) return;
    update();
    interval = setInterval(update, 1000);
  }

  function stop() {
    if (interval) {
      clearInterval(interval);
      interval = null;
    }
  }

  function changeFormat(newFormat) {
    format = newFormat;
    update();
  }

  return { start, stop, changeFormat };
}

// Uso
// const clockElement = document.getElementById('clock');
// const clock = createClock(clockElement, 'es-ES', 'compact');
// clock.start();
```

6. **Generador de logs con marcas de tiempo**:

```
class Logger {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.logs = [];
    this.maxLogs = 1000;
  }

  log(level, message, data = null) {
    const now = new Date();
    const timestamp = now.toLocaleTimeString(this.locale, {
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
      fractionalSecondDigits: 3,
      hour12: false
    });
```

```
    const logEntry = {
      timestamp,
      iso: now.toISOString(),
      level,
      message,
      data,
      full: `[${timestamp}] [${level.toUpperCase()}]${message}`
    };

    this.logs.push(logEntry);

    if (this.logs.length > this.maxLogs) {
      this.logs.shift();
    }

    console.log(logEntry.full);
  }

  info(message, data = null) {
    this.log('info', message, data);
  }

  warn(message, data = null) {
    this.log('warn', message, data);
  }

  error(message, data = null) {
    this.log('error', message, data);
  }

  getLogs() {
    return this.logs;
  }
}

const logger = new Logger('es-ES');
logger.info('Aplicación iniciada');
logger.warn('Memoria baja');
logger.error('Error al procesar', { code: 500 });
console.log(logger.getLogs());
```

7. **Cronómetro con formato local**:

```
class Stopwatch {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.startTime = null;
    this.elapsed = 0;
    this.running = false;
  }

  start() {
    if (this.running) return;
    this.startTime = Date.now() - this.elapsed;
    this.running = true;
  }

  stop() {
    if (!this.running) return;
    this.elapsed = Date.now() - this.startTime;
    this.running = false;
  }

  reset() {
    this.elapsed = 0;
    this.startTime = null;
    this.running = false;
  }

  getTime() {
    if (this.running) {
      this.elapsed = Date.now() - this.startTime;
    }

    const seconds = Math.floor(this.elapsed / 1000);
    const minutes = Math.floor(seconds / 60);
    const hours = Math.floor(minutes / 60);
    const remainingSeconds = seconds % 60;
    const ms = this.elapsed % 1000;

    const date = new Date(0, 0, 0, hours, minutes, remainingSecon
ds, ms);

    const options = {
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
```

```
      hour12: false
    };

    return {
      milliseconds: this.elapsed,
      formatted: date.toLocaleTimeString(this.locale, options),
      detailed: `${hours.toString().padStart(2, '0')}:${minutes.t
oString().padStart(2, '0')}:${remainingSeconds.toString().padStar
t(2, '0')}.${ms.toString().padStart(3, '0')}`
    };
  }
}

const timer = new Stopwatch('es-ES');
timer.start();
// ... después de un tiempo
timer.stop();
console.log(timer.getTime());
```

8. **Sistema de citas médicas**:

```
class AppointmentSystem {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.appointments = [];
  }

  scheduleAppointment(patient, doctor, date, duration = 30) {
    const start = new Date(date);
    const end = new Date(start.getTime() + duration * 60000);

    const timeOptions = {
      hour: '2-digit',
      minute: '2-digit'
    };

    const appointment = {
      patient,
      doctor,
      start,
      end,
      duration,
      timeDisplay: `${start.toLocaleTimeString(this.locale, timeO
```

```
ptions)} -${end.toLocaleTimeString(this.locale, timeOptions)}`,
      dateDisplay: start.toLocaleDateString(this.locale, {
        weekday: 'long',
        day: 'numeric',
        month: 'long',
        year: 'numeric'
      }),
      status: 'scheduled'
    };

    this.appointments.push(appointment);
    return appointment;
  }

  getAppointmentsByDate(date) {
    const targetDate = new Date(date).toDateString();
    return this.appointments
      .filter(a => a.start.toDateString() === targetDate)
      .sort((a, b) => a.start - b.start);
  }

  getTimeSlots(date, duration = 30) {
    const start = new Date(date);
    start.setHours(9, 0, 0, 0); // 9:00 AM
    const end = new Date(date);
    end.setHours(17, 0, 0, 0); // 5:00 PM

    const slots = [];
    let current = new Date(start);

    while (current < end) {
      const slotEnd = new Date(current.getTime() + duration * 600
00);
      const isBooked = this.appointments.some(a =>
        a.start.getTime() === current.getTime() &&
        a.start.toDateString() === date
      );

      slots.push({
        start: current,
        end: slotEnd,
        display: current.toLocaleTimeString(this.locale, {
          hour: '2-digit',
          minute: '2-digit'
```

```
        }),
        available: !isBooked
      });

      current = new Date(current.getTime() + duration * 60000);
    }

    return slots;
  }
}

const clinic = new AppointmentSystem('es-ES');
clinic.scheduleAppointment('Juan', 'Dr. Pérez', '2024-12-25T10:0
0:00');
console.log(clinic.getTimeSlots('2024-12-25'));
```

9. **Formateo de tiempo de duración**:

```
function formatDuration(seconds, locale = 'es-ES') {
  const hours = Math.floor(seconds / 3600);
  const minutes = Math.floor((seconds % 3600) / 60);
  const secs = Math.floor(seconds % 60);

  const date = new Date(0, 0, 0, hours, minutes, secs);

  const options = {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: false
  };

  let result = date.toLocaleTimeString(locale, options);

  // Si hay horas, mostrar formato HH:MM:SS
  if (hours === 0) {
    result = date.toLocaleTimeString(locale, {
      minute: '2-digit',
      second: '2-digit',
      hour12: false
    });
  }
```

```
  return result;
}

console.log(formatDuration(90)); // "01:30"
console.log(formatDuration(3661)); // "01:01:01"
console.log(formatDuration(45)); // "00:45"
```

10. **Recordatorios programados**:

```
class ReminderSystem {
  constructor(locale = 'es-ES') {
    this.locale = locale;
    this.reminders = [];
    this.checkInterval = null;
  }

  addReminder(message, date, sound = false) {
    const reminder = {
      message,
      date: new Date(date),
      formatted: new Date(date).toLocaleTimeString(this.locale, {
        hour: '2-digit',
        minute: '2-digit'
      }),
      sound,
      triggered: false,
      id: Date.now()
    };

    this.reminders.push(reminder);
    return reminder;
  }

  startChecking() {
    this.checkInterval = setInterval(() => {
      const now = new Date();
      this.reminders.forEach(reminder => {
        if (!reminder.triggered && reminder.date <= now) {
          this.triggerReminder(reminder);
        }
      });
    }, 1000);
  }
```

```
  triggerReminder(reminder) {
    reminder.triggered = true;
    console.log(`🔔 RECORDATORIO (${reminder.formatted}):${remind
er.message}`);
    if (reminder.sound) {
      // Reproducir sonido
    }
  }

  getReminders() {
    const now = new Date();
    return this.reminders
      .filter(r => !r.triggered && r.date > now)
      .sort((a, b) => a.date - b.date)
      .map(r => ({
        ...r,
        timeLeft: Math.floor((r.date - now) / 1000),
        timeLeftFormatted: this.formatTimeLeft(r.date - now)
      }));
  }

  formatTimeLeft(ms) {
    const seconds = Math.floor(ms / 1000);
    if (seconds < 60) return `${seconds} segundos`;
    if (seconds < 3600) return `${Math.floor(seconds / 60)} minut
os`;
    if (seconds < 86400) return `${Math.floor(seconds / 3600)} ho
ras`;
    return `${Math.floor(seconds / 86400)} días`;
  }
}

const reminders = new ReminderSystem('es-ES');
reminders.addReminder('Reunión importante', '2024-12-25T15:30:0
0');
reminders.addReminder('Tomar medicamento', '2024-12-25T14:00:0
0');
console.log(reminders.getReminders());
```

🧩 Relación con otros conceptos

- `Date.prototype.toLocaleDateString()`: Solo fecha localizada.
- `Date.prototype.toLocaleString()`: Fecha + hora localizada.

- `Date.prototype.toTimeString()`: Formato fijo del navegador.
- `Intl.DateTimeFormat`: API subyacente más poderosa.
- `Date.prototype.getHours()`: Obtener hora (0-23).
- `Date.prototype.getMinutes()`: Obtener minutos.
- `Date.prototype.getSeconds()`: Obtener segundos.
- `Date.prototype.getTimezoneOffset()`: Diferencia horaria.

🧠 Analogía

Imagina que `toLocaleTimeString()` es como un **traductor de horas**. Le das una hora en un "formato universal" (el objeto Date) y ella la traduce al formato que usa la persona en su país. En España, "15:30" se traduce como "15:30"; en EE.UU., se traduce como "3:30 PM"; en Alemania, como "15:30 Uhr".

📚 Fuente

- MDN Web Docs – Date.toLocaleTimeString()
- ECMAScript Specification – Date.prototype.toLocaleTimeString
- Intl.DateTimeFormat Specification
- Can I Use – Intl.DateTimeFormat